Project Proposal

Agentic AI Applications Course

Team Number:

Team Member Names: **Rashmi MD(R23EA096),Kiran S(R23EA058),K P Sree Rasswanth(R23EA053),C R Heemanth Reddy(R23EA029)**

Proposed Project Title: **Agentic AI-Based Legal Case Preparation System with Role-Based Access**

1. Problem Statement & Use Case:

What real-world problem are you solving?

Legal case preparation in India is time-consuming, document-heavy, and multilingual. Lawyers must manually:

- Read hundreds of pages of case documents
- Search relevant Acts, Sections, and Articles
- Identify applicable precedents
- Draft arguments and assess case strength

Citizens also struggle to understand their legal position due to complex legal language.

Who is the target user / beneficiary?

- Primary Users: Practicing Lawyers and Law Firms in India
- Secondary Users: Citizens seeking legal guidance

What currently happens?

Currently:

- Lawyers manually search judgments using platforms like Indian Kanoon

- Legal research takes hours or days.
- No structured AI system coordinates document reading, law mapping, precedent search, and strategy planning together.
- Existing legal research tools provide search but not autonomous reasoning or strategic planning.
- Citizens rely entirely on lawyers without preliminary structured understanding.

Pain Points:

- Multilingual document handling
- Manual research
- Time delay
- No structured AI-based legal planning
- Risk of missing important precedents

Why is this a good fit for an Agentic AI solution?

This problem requires:

- Multi-step reasoning
- Tool usage (OCR, translation, database search)
- Iterative analysis
- Precedent retrieval
- Strategy refinement
- Risk evaluation

A simple LLM prompt cannot:

- Process structured case memory
- Coordinate multiple legal tools
- Dynamically activate research agents based on user role
- Reflect and refine strategy
- Perform multi-agent legal reasoning

Therefore, this requires an **Agentic Multi-Agent System**, not just a chatbot.

2. Justify that your application is Agentic?

Core Agentic Capabilities Required

1. Tool Use
    - OCR tool
    - Translation tool
    - Vector search for precedents
    - Database retrieval
2. Planning C Decomposition
    - Break case preparation into stages
    - Identify sub-tasks: fact extraction → law mapping → precedent search → strategy planning
3. Multi-Agent Coordination
    - Separate agents for document processing, research, strategy, risk estimation
4. Reflection / Self-Correction
    - Detect inconsistent evidence
    - Re-query precedent database
    - Refine strategy if weak

What would fail if built as a single LLM call?

If implemented as one LLM call:

- No structured case memory
- No tool-based precedent retrieval
- No controlled role-based activation
- No iterative strategy improvement

- No evidence validation
- High hallucination risk

It would behave like a simple chatbot, not a legal reasoning system.

Expected Agentic Behavior Examples

- The system detects inconsistent witness statements and flags them.
- The precedent agent retrieves similar cases but finds weak similarity, then re-queries with refined keywords.
- The strategy agent revises arguments after risk estimation indicates low probability of success.
- The ethics agent flags missing documentary evidence.

3. High-Level Solution Overview

The system uses a multi-agent architecture where each agent performs a specialized legal task. A central coordinator (Orchestrator Agent) manages the workflow.

Role-based logic activates additional agents for lawyers.

Main Agents s Roles

- Document Processing Agent → OCR + Translation
- Case Understanding Agent → Extract facts C timeline
- Law Identification Agent → Map facts to legal sections
- Precedent Research Agent → Retrieve similar judgments
- Strategy Planning Agent → Suggest legal arguments
- Outcome Estimation Agent → Predict case strength
- Ethics C Fairness Agent → Validate risks
- Final Draft Agent → Generate structured report

- Supervisor Agent → Coordinate execution

Agentic Patterns Used

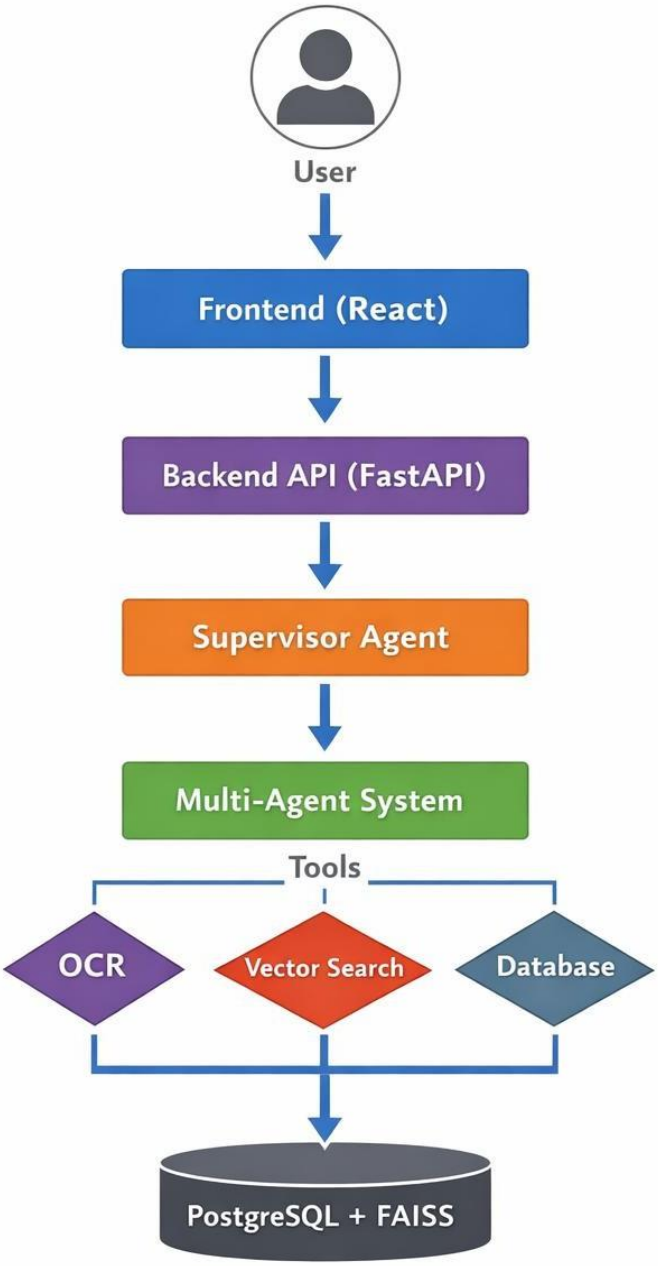
- Planning C Execution
- ReAct-style tool usage
- Multi-agent collaboration (CrewAI-style roles)
- Reflection loop
- Vector memory for precedent retrieval

Memory Strategy

- PostgreSQL for structured case data
- FAISS Vector DB for precedent embeddings
- Short-term session memory for workflow continuity

High-Level Flow

User Upload
→ Planner decomposes case
→ Document Agent processes
→ Case Agent extracts facts
→ Law Agent maps sections
→ Precedent Agent retrieves cases
→ Strategy Agent drafts arguments
→ Risk Agent evaluates
→ Reflection Loop checks
→ Final Report Generated


User
Frontend (React)
Backend API (FastAPI)
Supervisor Agent
Multi-Agent System
Tools
OCR
Vector Search
Database
PostgreSQL + FAISS

4. Proposed Technology Stack

LLM Provider

OpenAI (GPT-4o-mini or similar)

Reason:

- Strong reasoning ability
- Cost-efficient for student use
- Reliable API

Core Framework

CrewAI

Reason:

- Designed for multi-agent systems
- Easy role-based coordination
- Supports planning workflows

Tools

- Tesseract OCR (multilingual document reading)
- Translation API (IndicTrans / Google API)
- FAISS for vector search
- PostgreSQL database
- Custom law-section dataset

Vector Database

FAISS (Local)

Reason:

- Fast semantic search
- Free
- Good for precedent retrieval

Frontend / Delivery

React + FastAPI

Reason:

- Clean UI
- Industry standard
- Easy deployment

Evaluation Tools

- Manual evaluation rubric
- Accuracy comparison with known judgments
- Custom test case evaluation
- Token usage monitoring

Deployment Target

Frontend → Vercel
Backend → Render
Database → Supabase

5. *Success Metrics & Evaluation Plan*

Quantitative Metrics

- Task success rate ≥ 80% on 20 test cases
- Average response time < 10 seconds
- Precedent retrieval accuracy ≥ 75%
- Risk prediction consistency across repeated runs
- Token cost per case under defined budget

Qualitative Evaluation

- Legal reasoning accuracy (expert review)
- Output clarity (citizen version readability)
- Strategy coherence score (manual rubric)
- User satisfaction survey (5–10 participants)

Number of Test Cases

Planned:

- 20–30 legal case samples
- Civil, Criminal, Constitutional mix

Failure Modes

- Hallucinated legal sections
- Irrelevant precedents
- Overconfident risk prediction
- OCR errors

Measurement:

- Manual verification
- Cross-checking citations
- Error rate tracking

C. Risks, Challenges & Mitigation Plan

- Hallucination of laws
  → Cross-verification with structured law database
- Incorrect precedent similarity
  → Improve embedding tuning + similarity threshold
- High API cost
  → Limit token size + max iterations
- Infinite agent loops
  → Max iteration count in CrewAI
- Data privacy risk
  → Role-based access + encrypted passwords
- OCR inaccuracies
  → Manual correction override option

7. Timeline & Milestones

By Demo 1

- Single-agent document processing prototype
- Basic OCR + summary

By Document Submission

- Full architecture finalized
- Multi-agent design documented
- Basic precedent search implemented

By Demo 2

- Multi-agent MVP working
- Role-based dashboard
- Risk estimation integrated

By Final Submission

- Full workflow operational
- Evaluation metrics documented
- UI deployed
- Performance tested
- Final presentation prepared